

Personal Agent Platform

Diseño definitivo y documento de traspaso

Base técnica para construir en la PC principal una plataforma local de asistentes, automatizaciones, voz, correo, desarrollo de software e Infrastructure as Code.

Equipo de destino

**Intel Core i7-8700K | 32 GB RAM |
NVIDIA GTX 1060 6 GB**

Principio rector

El LLM propone. Las herramientas verifican. La persona autoriza.

1. Punto de partida

La implementación se realizará directamente en la PC principal. La notebook actual queda fuera del despliegue porque dispone de poco espacio libre. No existe todavía una instalación que deba migrarse: este documento define cómo iniciar correctamente el proyecto nuevo.

El repositorio infrastructure-as-code seguirá dedicado a las arquitecturas cloud. La plataforma personal tendrá un repositorio y un workspace independientes llamados personal-agent-platform.

Objetivos

- Operar n8n y modelos locales sin depender de un proveedor de IA para las tareas básicas.
- Permitir chat, voz, resúmenes de correo y automatizaciones personales.
- Crear agentes controlados para páginas web, código e Infrastructure as Code.
- Mantener secretos fuera de Git y aplicar mínimo privilegio.
- Poder trasladar el sistema a otro equipo sin rediseñar workflows.
- Preparar una separación futura entre plano de control e inferencia con GPU.

Fuera del MVP

- Conversación de voz full-duplex con interrupciones naturales.
- Varios modelos pesados ejecutándose simultáneamente.
- Base vectorial antes de demostrar que sea necesaria.
- terraform apply, merges o despliegues productivos autónomos.
- Acceso público directo a n8n, Ollama o PostgreSQL.

2. Arquitectura objetivo



Componente	Responsabilidad	MVP
n8n	Orquestación, schedules, estados, aprobaciones y notificaciones.	Sí
PostgreSQL	Persistencia de workflows, credenciales cifradas y ejecuciones.	Sí
Ollama	API local para el modelo de lenguaje.	Sí
Modelo 7B Q4	Chat técnico, código, Terraform y análisis.	Principal
Modelo 3B Q4	Alternativa rápida para pruebas o tareas sencillas.	Opcional
Whisper	Transcripción de voz.	Fase 2
TTS	Síntesis de respuestas habladas.	Fase 2
Runner aislado	Trabajo sobre repositorios y herramientas técnicas.	Fase 3
Reverse proxy/VPN	Acceso remoto protegido.	Después de local

Distribución inicial

PC principal

- Docker Compose
 - n8n
 - PostgreSQL
 - backup y health checks
- Ollama / llama.cpp
 - Qwen Coder 7B Q4
- Workspace controlado
 - repositorios autorizados
 - runners aislados

Prioridad de cola: voz > interacción manual > alertas > correo > código e IaC programados.

3. Estructura del repositorio

```
personal-agent-platform/  
  AGENTS.md  
  README.md  
  compose.yml  
  compose.override.example.yml  
  .env.example  
  config/  
    n8n/ models/ logging/  
  agents/  
    router/ assistant/ email/ repository/  
    architect/ implementer/ terraform/  
    security/ cost/ operations/  
  prompts/  
  schemas/  
  workflows/  
    chat/ email/ voice/ development/  
    infrastructure/ operations/  
  policies/  
  runners/  
  scripts/  
    backup/ restore/ healthcheck/ migrate/  
  tests/  
  docs/  
    architecture.md security.md  
    operations.md migration.md
```

Datos y secretos

Ubicación	Contenido	Git
Repositorio	Compose, prompts, schemas, políticas, scripts y documentación.	Sí
Docker volumes	PostgreSQL, datos de n8n y backups locales.	No
Almacén protegido	N8N_ENCRYPTION_KEY, OAuth tokens y secretos.	No
Cache recreable	Modelos descargados por Ollama.	No

Versionar el nombre y la versión del modelo, no los pesos. Fijar también versiones de imágenes Docker para que backup, restore y migración sean reproducibles.

4. Agentes y permisos

Los agentes son roles. Inicialmente compartirán el mismo modelo y se ejecutarán secuencialmente con prompts, herramientas y permisos diferentes.

Agente	Función	Permiso máximo
Router	Seleccionar workflow.	Lectura.
Conversacional	Chat y voz.	Lectura y memoria limitada.
Correo	Clasificar y resumir.	OAuth read-only.
Investigación	Buscar y citar fuentes.	Internet read-only.
Analista	Inventariar repositorios.	Repositorio read-only.
Arquitecto	Proponer diseños y riesgos.	Sin escritura.
Implementador	Modificar código aprobado.	Rama/worktree aislado.
IaC	Crear y revisar Terraform.	Plan, nunca apply autónomo.
Revisor	Revisar diff y pruebas.	Solo lectura.
Seguridad	Secretos, IAM y superficie.	Scanners read-only.
Costos	Cuotas y presupuesto.	Cloud read-only.
Operaciones	Alertas e incidentes.	Observabilidad read-only.
Despliegue	Operaciones predefinidas.	Wrapper y allowlist.

Un prompt de sistema no es una frontera de seguridad. Los límites efectivos deben estar en OAuth scopes, credenciales separadas, contenedores, wrappers, schemas, allowlists y aprobaciones.

5. Workflows previstos

Chat

entrada -> router -> contexto -> LLM -> schema -> respuesta

Control: Sin acciones externas en el MVP.

Correo

scheduler -> OAuth -> sanitizar -> clasificar -> resumir -> digest

Control: Read-only; el email es contenido no confiable.

Voz

push-to-talk -> STT -> router -> LLM -> TTS -> audio

Control: Prioridad alta y procesamiento por turnos.

Desarrollo

analizar -> plan -> rama -> editar -> tests -> review -> aprobar

Control: Runner sin Docker socket ni acceso completo al host.

Infrastructure as Code

diseño -> aprobar -> editar -> validate -> plan JSON -> políticas -> review

Control: Bloquear secretos, delete, replace, IAM amplio y costo incierto.

Operaciones

alerta -> correlacionar -> diagnosticar -> proponer -> notificar

Control: Diagnóstico read-only al comenzar.

6. Seguridad y validación

Nivel	Capacidad	Ejemplo	Aprobación
L0	Leer y responder	Consultar documentación	No
L1	Crear borradores	Respuesta de correo	Antes de enviar
L2	Escribir aislado	Modificar una rama	Antes de merge
L3	Operación predefinida	Desplegar SHA permitido	Según política
L4	Producción/destrutivo	apply, borrar o restaurar	Siempre

Controles obligatorios

- n8n, PostgreSQL y Ollama no se exponen directamente a Internet.
- PostgreSQL no publica su puerto fuera de la red interna de Compose.
- La clave de cifrado de n8n se respalda fuera del repositorio.
- n8n y los agentes generales no montan /var/run/docker.sock.
- Los runners reciben solo el repositorio, herramientas y credenciales necesarias.
- Emails, páginas, issues y documentos se tratan como entradas no confiables.
- Las salidas sensibles se validan mediante JSON Schema y campos desconocidos se rechazan.
- Acciones de envío, merge, despliegue o cloud requieren aprobación.

Validadores deterministas

Dominio	Validaciones
Código	Tests, linters, build, análisis estático y diff.
Terraform	fmt, init -backend=false, validate, plan -json y políticas.
Seguridad	Secret scanning, dependencias, IAM, rutas y red.
Costos	Cuotas reales, fuentes oficiales y presupuesto máximo.
Despliegue	Allowlist, SHA inmutable, digest, health check y rollback.

7. Fases de implementación

Fase	Alcance	Criterio de salida
0. Repositorio	AGENTS.md, amenazas, schemas y documentación.	Reglas y límites revisados.
1. Base	Compose, PostgreSQL, n8n, Ollama y health checks.	Reinicio sin pérdida de datos.
2. Chat	Router mínimo y respuesta estructurada.	Conversación local estable.
3. Resiliencia	Backup, restore, logs y alertas.	Restauración probada.
4. Correo	OAuth read-only y resumen programado.	Digest sin acciones de escritura.
5. Voz	Push-to-talk, STT y TTS.	Conversación por turnos.
6. Desarrollo	Runner, ramas, tests y review.	Cambios aislados y reproducibles.
7. IaC	Plan JSON, políticas de costos y seguridad.	Sin apply y con bloqueo destructivo.
8. Operaciones	Alertas y despliegues restringidos.	Auditoría y rollback probado.

Criterios del MVP

- n8n y PostgreSQL sobreviven a un reinicio.
- El chat invoca Ollama y valida la estructura de la respuesta.
- El modelo puede cambiarse mediante configuración.
- Health checks y errores producen notificaciones claras.
- El backup se restaura en una instalación limpia.
- No hay secretos versionados ni puertos internos expuestos.
- La concurrencia del LLM se limita a un trabajo pesado.

La primera implementación debe cubrir únicamente las fases 0 a 3. Correo, voz y escritura sobre repositorios comienzan después de probar restore.

8. Operación, backup y evolución

Recursos iniciales

Servicio	Límite inicial
PostgreSQL	1-2 GB de RAM; volumen persistente.
n8n	2 GB de RAM; concurrencia controlada.
LLM	Un modelo 7B Q4 activo; contexto moderado.
Workers	Un workflow pesado por vez.
STT/TTS	Desactivados hasta la fase de voz.

Backup mínimo

- pg_dump de PostgreSQL con verificación de integridad.
- Copia cifrada de N8N_ENCRYPTION_KEY y secretos necesarios.
- Exportación adicional de workflows como defensa en profundidad.
- Retención limitada y prueba periódica de restauración.
- Los pesos de modelos se vuelven a descargar desde una lista versionada.

Evolución con ThinkCentre

ThinkCentre siempre encendido

- n8n, PostgreSQL, colas, correo, VPN y monitoreo

PC principal con GPU

- Ollama, STT, TTS y runners de código

Solo cambian URLs de servicio:

OLLAMA_BASE_URL / STT_BASE_URL / TTS_BASE_URL / RUNNER_BASE_URL

Una futura GPU de 12 GB permitirá modelos 14B cuantizados y mayor contexto. La arquitectura, los permisos y las aprobaciones deben permanecer iguales.

9. Prompt para iniciar el nuevo proyecto

Usar el siguiente mensaje como solicitud inicial en el nuevo workspace:

Quiero crear un nuevo repositorio llamado personal-agent-platform.

Hardware de destino:

- Intel Core i7-8700K
- 32 GB de RAM
- NVIDIA GTX 1060 de 6 GB
- Windows con WSL2/Ubuntu y Docker

Objetivo inicial:

Construir la base local y portable para n8n, PostgreSQL y Ollama.

La primera iteracion debe cubrir solo repositorio, seguridad, chat textual, health checks, backup y restauracion. No implementar aun correo, voz, agentes con escritura ni despliegues.

Requisitos:

- analizar primero el entorno y las versiones instaladas;
- crear AGENTS.md antes de implementar;
- usar Docker Compose con versiones fijadas;
- no almacenar secretos en Git;
- mantener PostgreSQL y Ollama sin exposicion publica;
- no montar el Docker socket en n8n;
- usar variables para todas las URLs de servicios;
- limitar a una inferencia pesada simultanea;
- incluir .env.example, health checks, backup y restore;
- documentar migracion a otra computadora;
- no ejecutar comandos destructivos sin aprobacion.

Antes de modificar archivos:

1. proponer la arquitectura concreta;
2. indicar archivos a crear;
3. identificar riesgos y consumo de recursos;
4. mostrar el modelo de secretos y persistencia;
5. esperar mi aprobacion.

Archivos que deben acompañar este PDF

- Este PDF de diseño y traspaso.
- El PDF anterior de hoja de ruta, como referencia ampliada opcional.
- Ningún .env, token, clave o credencial de la instalación futura.

10. Checklist de arranque en la PC principal

Antes de crear el repositorio

- Confirmar espacio libre, salud del disco y estrategia de backup.
- Actualizar driver NVIDIA y verificar acceso a GPU desde WSL2.
- Instalar Docker y comprobar Docker Compose v2.
- Definir una carpeta dentro del filesystem de WSL, no bajo /mnt/c.
- Decidir si el acceso inicial será solo localhost o también VPN privada.

Durante la implementación

- Fijar versiones y revisar cada imagen antes de ejecutarla.
- Crear secretos localmente y excluirllos mediante .gitignore.
- No habilitar workflows con escritura hasta probar backup y restore.
- Registrar decisiones arquitectónicas en docs/architecture.md.
- Medir RAM, VRAM, latencia y temperatura con 3B y 7B antes de elegir modelo.

Definición de éxito

La primera etapa termina cuando una instalación limpia puede levantarse desde el repositorio, recibir una pregunta, responder mediante Ollama, registrar la ejecución, reiniciarse sin perder datos y restaurarse desde un backup probado.

Referencias técnicas

n8n - Docker Compose:

<https://docs.n8n.io/deploy/host-n8n/install-options/install-using-docker-compose.md>

n8n - Ollama Chat Model:

<https://docs.n8n.io/integrations/builtin/cluster-nodes/sub-nodes/n8n-nodes-langchain.lmchatollama/>

Ollama - Qwen2.5-Coder:

<https://ollama.com/library/qwen2.5-coder/tags>

whisper.cpp:

<https://github.com/ggml-org/whisper.cpp>